

SpaceLift — PRD package

Sep 26, 2026 · @Evin Ealias

Summary

SpaceLift turns one photo of a renter's real room into an edited image of that room plus a priced, shoppable list of small, lease-safe changes, from \$0 to \$500. The image and the list are one linked object: every item is a numbered pin in the render and a row in the list, and every change updates both.

Problem. Renters in small spaces get inspiration from photos of other people's rooms, then guess at sizes, prices and whether an item will cost them their deposit. Nothing shows the change in *their* room, at *their* budget, under *their* lease.

Goals for v1

- Render the user's own room with new items placed, in under 60 seconds for a first design.
- Make every total add up from real item prices, and label every price as an estimate with a checked date.
- Make lease safety visible: one install badge per item, filtered by the user's Rules tier.
- Make refinement cheap: a single-item change re-renders only that item's region, in under 15 seconds.
- Lead with the \$0 Rearrange mode and the "Same room, three renters" compare view as the demo moments.

Non-goals for v1

- Large furniture, appliances and construction (redirected to small-lift alternatives).
- In-app checkout, AR, a community feed.
- Exact product reproduction in the render; the render shows a close match, and the Buy link is the source of truth.

Users and key scenarios

The primary user is a renter aged 22–38 in a small apartment or studio, spending \$0–\$500. The MVP covers the living room only and sells from Amazon only; other rooms and stores come in Next. Desktop web is the primary layout; mobile is a full, single-column

experience.

Scenario	Input	What success looks like
Host a party	Living room photo, 30th birthday, 12 guests, gold/blush/white, \$75 hard cap, no damage	9-item plan at \$73.17, all lease-safe, evening view shows the lights on, a setup and takedown checklist
Rearrange what I own	Living room photo, \$0, "I'll have help", mounted TV marked "Can't move"	Same furniture in new spots, a 6-step move plan with effort and time, doors and walkways checked
Refresh a room	Living room, \$150, Minor changes OK	A plan with a few amber items, and a landlord letter for the one item that needs approval
Torn between budgets	One photo, three presets	Three plans side by side with a comparison table, then "Continue with this plan"
Tweak one thing	"@String lights can they be black wire?"	Only pin 5 re-renders, price and list row update, a new version entry appears

Scope and release plan

MVP ships the full photo-to-plan loop, lease-safe badges, lighting moods with the evening view, \$0 Rearrange and the three-renters compare. Everything that needs accounts, extra data sources or legal review waits for Next.

Area	MVP	Next	Out of scope
Intake	Photo upload, one-wall measurement, guided cards, Brief bar	Multiple photos per room, voice input	—
Design	Render, before/after, pins with two-way highlight, Day/Evening, version history	"Point to it" region select	AR

Area	MVP	Next	Out of scope
List	Thumbnail, store, price, Buy link, variants, swap drawer, budget meter, lease-safe badges and summary, fit indicators	Group by store and light layer, Fit to budget, "I already have this", budget breakdown	In-app checkout
Modes	\$0 Rearrange with move plan and arrows	"Finish it for under \$25" add-ons	Large furniture
Renter tools	Rules tiers and badge filtering	Landlord letter with status tracking	Legal advice
Compare	Three renters preset, 3 columns, synced slider and Day/Evening, table, Continue	Budget ladder, Mood trio, Custom, Presenter mode, Unique highlight, mobile carousel	—
Account	Share link, anonymous projects saved to device	Saved projects, accounts	Community feed

MVP boundaries: living room only, Amazon only, and an AI-built catalog of small items with no furniture (see Product catalog). Other rooms and stores move to Next.

The canvas mockups are the visual reference for every row above; the component sheet is the source for pills, chips, pins, item cards, badges and meters.

Functional requirements

Each requirement has an ID for tickets and QA. Priority is MVP unless marked Next.

Intake

- **IN-1** Order: Photo → Measurement → Plan → Room → Budget → Mood → Style (optional) → Rules → Stores → Summary. Every step is skippable; a single typed message can answer several steps. In MVP the Room step is fixed to Living room and the Stores step is hidden (Amazon only).
- **IN-2** A full-width strip under the app bar shows the 10-step progress bar first, then the "Your plan so far" pills.
- **IN-3** Photo quality check runs on upload and warns for too dark, blurry or tightly cropped photos, with retake tips.

- **IN-4** Measurement: auto-highlight the wall facing the camera, draggable line, ft/in ↔ cm, optional ceiling height, Small ~8 ft / Medium ~12 ft / Large ~16 ft chips. Skipped → results carry a "Sizes estimated" tag.
- **IN-5** Room type is detected from the photo and confirmed with chips.
- **IN-6** Budget slider snaps to \$0, \$25, \$50, \$100, \$300, \$500, plus type-in and Hard cap / Flexible ±10%. \$0 switches to Rearrange mode with a toast.
- **IN-7** Mood cards: Cozy (2200–2700K), Sleek (3000–3500K), Minimalist (2700–3000K), Festive (warm + color). Each shows the same sample room lit differently and an ambient / task / accent mix bar. Pick one.
- **IN-8** Rules tiers: No damage, Minor changes OK, Anything goes; toggles for keep furniture (default on), pet-friendly, kid-safe, portable.
- **IN-9** Summary card with Edit, "Design it" and "Show me 3 ways".

Workspace and chat

- **WS-1** Desktop: chat pane ~38% (min 380 px) and output pane, resizable. Full-width Brief bar under the app bar; tapping a pill opens an inline editor; a change after generation shows "Update design?".
- **WS-2** Result message: 2–3 line summary plus suggestion chips; out-of-scope or over-budget asks get an alternative card, never a flat refusal.
- **WS-3** @-mentions autocomplete list items; clicking a list item inserts an item chip; chips in messages highlight the item.
- **WS-4** Each generation adds a version entry ("v3 · lights → black wire · \$73.17") with View; filmstrip supports Restore and Compare with current.
- **WS-5** Next: "Point to it" box or tap on the image becomes a region chip in the composer.

Design canvas

- **DC-1** Before / Slider / After / Side-by-side; zoom, pan, fullscreen, download.
- **DC-2** Numbered pins match list numbers; Show pins toggle; pins are keyboard focusable.
- **DC-3** Selecting from a pin, row or chat chip pulses the pin, outlines the item, dims the rest ~40% and scrolls the row into view; Esc clears.
- **DC-4** Day / Evening segmented control, prominent on the image; Evening shows the new lights on.

- **DC-5** Generating states: full generation keeps the last image under a shimmer with 4 named steps; single-item edit shimmers only that region with a spinner on its pin.

Shopping list

- **SL-1** Item card: number, thumbnail, name, one-line why, store, price marked est., quantity, one install badge, variants, fit indicator, Buy / Swap / Lock / Remove / I already have this.
- **SL-2** Variant change updates thumbnail and price instantly and re-renders only that item.
- **SL-3** Swap drawer: 3–6 alternatives with price difference, filters Cheaper / Same store / Different style, Preview in room, Use this.
- **SL-4** Locked items survive regenerations. "I already have this" sets cost to \$0 with an Owned tag (Next).
- **SL-5** Price honesty: "Prices are estimates, checked [date]"; no exact link → "Find similar at [store] ↗"; broken link state.
- **SL-6** Budget meter always visible: green under, amber within 10%, red over, each with text and icon. Over → "You're \$X over. Fit to budget?" with a cost diff to accept (Next).
- **SL-7** Lease-safe summary line and filter chips All / No damage / Minor / Needs OK.

\$0 Rearrange

- **RA-1** After the photo, detected furniture is outlined; tapping marks it "Can't move" (lock). Lifting chips: alone / I'll have help / light items only.
- **RA-2** Output keeps the same furniture in new positions, adds nothing, and swaps the list for a Move plan: step, thumbnail cropped from the photo, from → to, 1- or 2-person, time, tip.
- **RA-3** Before view shows dashed numbered arrows (Show moves). Layout checks: doors clear, walkways ≥ 3 ft, lamps near an outlet. "Why this works" card. Meter reads "\$0 spent".

Lease safety and landlord letter

- **LS-1** Every product carries exactly one install badge: green (Freestanding, Adhesive, Tension rod, Plug-in or battery, Peel-and-stick), amber (Small nail hole, Swaps a bulb), plum (Needs landlord OK). Tooltip carries a removal tip.
- **LS-2** No damage shows green only; Minor adds amber; Anything goes shows all.

- **LL-1** (Next) Letter drawer: upgrades to include, Friendly / Formal, Email / Text / Portal, names and unit, optional lease clause, four promise toggles, attach design image, editable draft, Copy / Open in email / Regenerate, not-legal-advice line.
- **LL-2** (Next) Status Not asked → Asked → Approved / Declined, set by the user; Declined triggers a lease-safe swap suggestion.

Compare view

- **CV-1** Three columns with plan header, render, meter "totaled from item prices", lease summary, lighting mix, item strip, what's different, Continue and Edit.
- **CV-2** Synced Before/After slider, Day/Evening and pins across columns; columns render in parallel and appear as each finishes.
- **CV-3** Comparison table: Total, Left in budget, Items, Lease-safe / Minor / Needs OK, Lighting mix, Stores used, Est. setup time; best cell marked with a label, not color alone.

Mobile

- **MB-1** Bottom tabs Chat / Design / List, floating budget pill, pickers as bottom sheets, swipe rows for Swap and Remove, chat item chips jump to Design with the item highlighted.

Image generation

Plan first, pixels second: a structured plan (items, prices, positions in the photo) is decided before any image is generated, and the model only edits masked regions. Everything outside the masks stays the user's original pixels, so the room is always recognizably theirs and pins, list rows and totals come from the same data.

flowchart LR

```

A[Upload + quality check] --> B[Scene understanding]
B --> C[Planner + catalog]
C --> D[Budget + fit validator]
D --> E[Placement + masks]
E --> F[Masked edit render]
F --> G[Verify each item]
G -->|fail, max 2 retries| F
G --> H[Composite + pins]
H --> I[Evening relight]
H --> J[Price + link refresh]

```

Stages

#	Stage (UI step)	What happens	Output
1	Ingest	Fix EXIF orientation, strip GPS, resize to 2048 px long edge. Quality check: brightness histogram (too dark), Laplacian variance (blurry), detected floor + wall coverage (too tight)	Normalized image, quality flags
2	Scene understanding ("Reading your room")	Vision-language model gives room type, style and light conditions. Open-vocabulary segmentation gives wall, floor, ceiling, windows, doors, outlets and furniture instances as masks. Monocular depth + normals give the back-wall and floor planes and an approximate camera	SceneGraph JSON, cached by photo hash
3	Scale	The user's wall width sets pixels-per-foot on the wall plane; floor scale comes from depth. No measurement → room-type default and "Sizes estimated"	Scale per plane
4	Planning ("Picking items under \$75")	LLM planner with tools: catalog search filtered by stores, badge tier, fit and budget; the mood's lighting recipe sets lighting's share first. It proposes items, variants, quantities, anchor surfaces and a one-line why each	Draft plan

#	Stage (UI step)	What happens	Output
5	Validate	Deterministic code, not the model, checks: total $\leq$ cap (or $\leq$ cap $\times$ 1.1 when flexible), one badge per item within the tier, dimensions fit the wall/floor, no large furniture, locked items kept. Violations go back to the planner with the reason	Final plan
6	Placement ("Placing them")	For each item: project real dimensions onto its anchor plane, get a pixel quad, avoid furniture masks, doors and a 3 ft walkway, snap lights near outlets. Build a feathered mask per item	Boxes, masks, pin points
7	Render	One masked edit pass over the union of masks, conditioned on the original photo, a per-item text prompt and the product's reference image. Output is composited so pixels outside the dilated masks are the original	Day render
8	Verify	Per item: grounding check that the item appears in its box and resembles the product image; outside the masks, pixel difference below threshold; no new people, text, logos or large furniture. Failed items re-render alone, up to 2 times, then show as "approximate"	Verified render

#	Stage (UI step)	What happens	Output
9	Finish ("Finding where to buy")	Pins at each verified item's centre; highlight regions stored; links resolved and prices refreshed	Version record

Prompting

Prompts are built from the plan, never free text from the user. The user's words shape the plan; the plan shapes the prompt. Template per item:

```
Place {product_name} ({variant}), about {w_in} x {h_in} in, on the {anchor}
at the marked area.
Match the photo's perspective, lens and existing light. {install_note}.
Do not change anything outside the mask. No people, text or logos.
```

Global suffix: the room's detected style and light direction from the SceneGraph, so shadows match the window.

Edits after the first design

Edit	Regions re-rendered	Target p50
Variant change (white → black wire)	That item's mask only, with the new variant's reference image	8 s
Swap or add one item	Union of the old and new item masks, dilated	12 s
"Point to it" region	The user's box becomes the mask	12 s
Full regeneration ("More festive")	All unlocked items; locked items keep their pixels	45 s

Every edit writes a new version; nothing overwrites an old one, so Restore and Compare are free.

Evening view

Evening is derived from the day render, not generated from scratch. A deterministic pre-pass lowers exposure, shifts white balance cooler and turns windows to night. A low-strength relight pass then turns on each lighting item from the plan at its colour temperature (for example 2700K for Cozy string lights, colour bulbs for Festive), guided by depth and edges so geometry cannot move. The result is cached per version and generated in the background right after the day render, so the toggle is instant.

\$0 Rearrange

- The planner works on a top-down floor plan built from the floor plane and furniture footprints, and runs the door, walkway and outlet checks there.
- Where a piece leaves, the background is inpainted into a clean floor and wall.
- Pieces that only slide are cut from the photo and re-projected at the new depth, keeping the user's real pixels.
- Pieces that turn (the sofa facing the window) need unseen sides. These are generated with the user's own crop as the reference and labelled "preview" in the UI.
- Movement arrows are drawn by the frontend from the old and new centroids; the renderer never draws them.

Compare and demo assets

The three compare columns share stages 1–3 through the SceneGraph cache and run stages 4–9 in parallel; each column streams in when done. Intake mood cards and landing examples are pre-rendered static assets, not generated per user.

Model strategy

Each role sits behind an adapter so vendors can change without touching product code: vision-language understanding, open-vocabulary segmentation, monocular depth, masked image editing with image references, relighting, and grounding for verification. Before launch, run a bake-off of 2–3 candidates per role on a 200-photo evaluation set of real small apartments, scored on item fidelity, room preservation, lighting realism and latency.

Safety

Reject photos with explicit content; blur faces and documents before storage; never render people, text or brand logos; keep all edits inside masks.

Backend design

A stateless API in front of a durable job system: every generation is a job with named steps that stream progress to the client, and every result is an immutable version. The catalog is its own service, because prices and links change on their own schedule.

```

flowchart LR
    Web[Web + mobile web] -->|REST + SSE| API[API service]
    API --> Conv[Conversation service]
    API --> Jobs[Job orchestrator]
    Conv --> Jobs
    Jobs --> Vision[Vision workers]
    Jobs --> Render[Render workers GPU]
    Jobs --> Cat[Catalog service]
    Conv --> Cat
    API --> PG[(Postgres + pgvector)]
    Jobs --> PG
    Render --> S3[(Object storage + CDN)]
    Cat --> PG
    Cat --> Feeds[Retailer feeds and APIs]
  
```

Services

Service	Responsibility	Notes
API service	Projects, briefs, versions, items, share links, auth (device token in MVP, accounts in Next)	Stateless, horizontally scaled; idempotency keys on every write
Conversation service	Turns chat into typed actions with an LLM tool loop: update_brief, edit_item, swap_item, add_item, regenerate, fit_to_budget, draft_letter, compare	The LLM never writes prices or totals; tools return them from the catalog
Job orchestrator	Durable workflows for generate, edit, relight, rearrange, compare; retries per step; progress events	Priority: single-item edit > full generation > compare > evening prefetch
Vision workers	Quality check, segmentation, depth, SceneGraph, verification	SceneGraph cached by photo hash

Service	Responsibility	Notes
Render workers	Masked edits, relight, clean-plate inpainting, compositing	GPU pool with autoscaling on queue depth; model adapters
Catalog service	Product ingestion, normalization, search, price refresh, link checks	See below
Letter service (Next)	Landlord letter drafts from template + plan	Never gives legal claims; fixed disclaimer

Catalog and pricing

- **Sources:** Amazon only in MVP, through the Creators API and an AI enrichment pipeline (see Product catalog). Other stores become connectors in Next.
- **Normalized product:** store, SKU, title, URL, images, price, currency, checked_at, variants (color, size, each with its own image and price), dimensions, category, light layer, colour temperature, install method.
- **Install badge:** assigned by rules on category and attributes, with an LLM classifier for the rest and human review for anything amber or plum. Each badge has a removal tip.
- **Search:** filters first (store, badge tier, price ceiling, dimensions), then vector similarity on title + image embeddings for style.
- **Freshness:** prices refresh nightly for items in active projects and on view when older than 24 hours; broken links trigger an "unavailable" state and a suggested swap.

Progress and real time

A job emits events `step_started`, `step_done`, `item_ready`, `version_ready` and `failed` over Server-Sent Events. The client maps the four step names to the canvas shimmer, and compare columns render as each `version_ready` arrives.

Storage and retention

- Postgres for projects, versions, items and catalog; pgvector for embeddings.
- Object storage for originals, masks and renders, served through a CDN with signed URLs.
- Originals and renders of anonymous projects deleted after 30 days of inactivity; users can delete a project and its images at any time.

Abuse and limits

Per-device rate limits on full generations, a daily GPU budget alarm, and upload checks for file type and size (max 20 MB).

Product catalog (MVP)

For MVP an AI pipeline builds and maintains the catalog: living room only, Amazon only, small items only. Every record passes code checks and, where it matters for a renter's deposit, a human spot-check before the planner can pick it.

What's in and out

Category	Examples	Default install badge	Light layer
Lamps	Table, floor, desk and clip lamps (plug-in)	Plug-in or battery	Ambient or task
String and accent lights	Fairy, café and curtain lights, LED strips, warm or smart bulbs	Plug-in or battery; Swaps a bulb	Accent
Rugs	Area rugs up to 8×10, runners, rug pads	Freestanding	—
Wall art	Posters, prints, framed art, canvas paintings	Adhesive (strips) or Small nail hole	—
Mirrors	Leaning and adhesive-mount mirrors	Freestanding or Adhesive	—
Textiles	Pillow covers, throws, curtains, tension rods, slipcovers	Freestanding or Tension rod	—
Plants	Faux plants and stems, planters	Freestanding	—
Decor objects	Vases, LED candles, trays, bookends, small sculptures	Freestanding	Accent (candles)
Storage accents	Baskets, bins, small wall ledges	Freestanding or Small nail hole	—

Category	Examples	Default install badge	Light layer
Party and seasonal	Balloons, garlands, backdrops, tableware	Adhesive or Freestanding	—
Hanging hardware	Adhesive hooks and strips	Adhesive	—

Excluded: sofas, couches, chairs, all tables, beds, dressers, bookcases, TV stands, appliances, hardwired fixtures and anything over \$500. The exclusion is enforced twice: by an allowlist of Amazon browse nodes, and by a classifier that rejects furniture that slips into an allowed node.

How the AI builds it

1. **Seed queries.** An agent writes search queries per category × mood (Cozy, Sleek, Minimalist, Festive) × price band, e.g. "battery warm white fairy lights 20 ft".
2. **Retrieve.** Search and item lookups through Amazon's Creators API.
3. **Hard filters (code).** Allowed browse node, price ≤ \$500, in stock, minimum rating and review count, no excluded keywords.
4. **Enrich (AI).** From title, bullet points and images: clean name, category, dimensions (parsed, or estimated from the image and flagged), install method → one badge + removal tip, light layer, colour temperature, colours, style tags, best-fit moods, a short "why" line, and variants grouped under one product.
5. **Render reference.** Pick the cleanest image and cut the product out for render conditioning.
6. **Validate.** Code checks the schema and that badge, category and size agree; a second model pass reviews and gives a confidence score.
7. **Human spot-check.** Every amber or plum badge, every low-confidence record, plus a random 5% sample.
8. **Publish.** Index filters and embeddings for the planner's search.
9. **Refresh.** Price and availability re-checked on a schedule; unavailable items are hidden and swaps suggested in open projects.

Coverage targets

The proposed starter size is about 1,500 items, roughly 150 per category. It should have at least 10 items for every category × mood pair and cover four price bands: under \$10, \$10–25, \$25–50, over \$50. The \$75 birthday sample then has enough choice for Fit to budget and the swap drawer.

Example record

```
{
  "asin": "B0XXXXXXX",
  "title": "Warm-white fairy lights, 20 ft, battery",
  "category": "string_accent_lights",
  "price": 10.00, "currency": "USD", "price_checked_at": "2026-09-26T08:00:00Z",
  "variants": [{"wire": "white"}, {"wire": "black"}, {"wire": "copper"}],
  "dimensions_in": {"length": 240}, "dimensions_source": "listing",
  "install_method": "battery", "badge": "plug_in_or_battery", "removal_tip":
  "Unhook and lift off; nothing to patch.",
  "light_layer": "accent", "kelvin": 2700,
  "moods": ["cozy", "festive"], "styles": ["boho", "colorful"],
  "render_ref_url": "s3://catalog/refs/B0XXXXXXX.png",
  "confidence": 0.93, "review": "auto_approved"
}
```

Amazon data access

This is the MVP's biggest dependency. PA-API 5.0 was retired on May 15, 2026 and replaced by the Creators API, which returns up to 10 items per search or lookup call ([dev.to](#), [Amazon](#)). Access reportedly needs an accepted Amazon Associates account with at least 10 qualifying sales in the last 30 days, so a brand-new app may not have API access on day one.

- **Bootstrap:** the team picks the starter ASINs by hand and creates Associates links for them. The AI then enriches each one and runs the same validation. Only the automated retrieval (step 2) waits for API access.
- **No scraping** of Amazon pages; everything comes from the API or from the team's manual picks.
- **Disclosure:** Buy links are Associates links; the list footer says SpaceLift may earn a commission.

- **Storage rules:** how long prices and images may be cached must be confirmed against the Associates Operating Agreement before launch.

Data model and API

A project holds one photo and a brief; each generation is an immutable version holding its render, pins and priced items. Items reference catalog products but copy the price at generation time, so old versions keep their totals.

Core entities

Entity	Key fields
Project	id, owner (device or user), name, mode (standard, rearrange), current_version_id, created_at
Photo	id, project_id, original_url, width, height, quality_flags, scene_graph_id
SceneGraph	id, photo_hash, room_type, surfaces[], furniture[] (mask, footprint, can_move), outlets[], scale_px_per_ft, measured (bool)
Brief	project_id, plan_type, occasion details, room, budget, cap_type, mood, style, palette, avoid_colors, rules_tier, toggles, stores[], prefer_cheapest, wall_width_in, ceiling_in
Version	id, project_id, number, parent_version_id, change_summary, day_render_url, evening_render_url, total, status, created_at
VersionItem	version_id, pin_number, product_id, variant_id, qty, unit_price, price_checked_at, badge, why, box, mask_url, locked, owned, fit (fits, tight, too_big), render_quality (verified, approximate)
MoveStep	version_id, step, furniture_id, from_box, to_box, people (1, 2), minutes, tip
Product / Variant	store, sku, title, url, images, price, checked_at, dimensions, category, light_layer, kelvin, install_method, badge, removal_tip

Entity	Key fields
Message	project_id, role, content, item_refs[], region_ref, version_id
LandlordRequest (Next)	project_id, item_ids[], tone, channel, promises, draft, status (not_asked, asked, approved, declined)
CompareSet	id, photo_id, preset, plan version_ids[3]

API (REST + SSE)

Method	Path	Purpose
POST	/projects	Create a project; returns an upload URL
POST	/projects/{id}/photo	Register the uploaded photo; starts quality check and scene understanding
PATCH	/projects/{id}/brief	Update any brief field; returns "update design?" hint if a version exists
POST	/projects/{id}/generate	Start a full generation; returns job_id
POST	/projects/{id}/messages	Send a chat message; may return a job_id
POST	/versions/{id}/items/{pin}/variant	Change a variant; single-item job
GET	/versions/{id}/items/{pin}/alternatives	Swap drawer options with price difference
POST	/versions/{id}/items/{pin}/swap	Use or preview an alternative
PATCH	/versions/{id}/items/{pin}	Lock, owned, qty, remove
POST	/versions/{id}/fit-to-budget	Proposed swaps and cost diff (Next)
POST	/versions/{id}/restore	Make an old version current (creates a new version)
POST	/compare	Start three plans from one photo and a preset
POST	/projects/{id}/landlord-letter	Draft a letter (Next)

Method	Path	Purpose
POST	/versions/{id}/share	Create a read-only share link
GET	/jobs/{id}/events	SSE progress stream

Non-functional requirements

The targets below are proposed starting points for engineering to confirm after the model bake-off.

Area	Requirement
Latency	First design $p50 \leq 45$ s, $p95 \leq 90$ s; single-item edit $p50 \leq 12$ s; evening toggle instant after prefetch; first progress event ≤ 2 s
Correctness	Totals always computed from item prices by code; budget cap never exceeded on hard cap; one install badge per item
Room fidelity	Pixels outside item masks match the original (mean difference below a set threshold)
Cost	Track GPU and model cost per design and per edit from day one; set a per-design ceiling after the bake-off
Availability	99.5% for the API; generation failures show "Something went wrong — try again" with the previous version intact
Accessibility	WCAG 2.1 AA contrast; everything keyboard reachable including pins and list rows; status always text + icon, never color alone; text on sage uses plum-charcoal
Privacy	Photos private by default; GPS stripped; faces and documents blurred; delete project deletes images; share links are unguessable and revocable

Area	Requirement
Honesty	Every price labelled est. with a checked date; "approximate" label when an item couldn't be verified in the render; rearranged pieces with imagined sides labelled preview
Browsers	Latest two versions of Chrome, Safari, Firefox, Edge; mobile Safari and Chrome

Success metrics

The north-star metric is the share of first designs where the user clicks at least one Buy link within 7 days. Targets get set after two weeks of baseline data.

Metric	Definition
Activation	Uploads that reach a first design
Intake completion	Users who finish intake / users who upload
Buy intent (north star)	First designs with ≥ 1 Buy click within 7 days
Refinement	Edits per project; share of edits that are single-item
Trust	Share of items rendered "verified" vs "approximate"; reports of wrong price or broken link
\$0 mode	Rearrange projects started from landing; move plans marked done
Compare	Compare sessions that end in "Continue with this plan"
Speed	Time to first design, p50 and p95

Events: photo_uploaded, quality_warning_shown, intake_step_completed (step, skipped), design_started, design_ready (ms, total, item_count), item_selected (source: pin, row, chip), variant_changed, swap_opened, swap_used, buy_clicked (store, price),

evening_toggled, version_restored, compare_started, compare_continued (plan), letter_drafted, landlord_status_changed, share_created.

Risks, open questions and milestones

The biggest risk is render fidelity: if placed items don't look like the products users buy, the list loses trust. The plan-first pipeline, per-item verification and the "approximate" label are the mitigations.

Risks

Risk	Mitigation
Rendered item differs from the real product	Reference-image conditioning, grounding check, retry, "approximate" label
Room changes outside the edited areas	Composite original pixels outside masks; pixel-diff check
Stale prices or dead links	Nightly refresh, on-view refresh after 24 h, "Find similar" fallback
No Amazon API access at launch	Hand-picked starter ASINs enriched by AI; automated retrieval switches on once Creators API access is granted
Wrong install badge (deposit risk)	Rules + classifier + human review for amber and plum; removal tips from a reviewed library
Rearrange renders look fake when pieces rotate	Prefer slides over rotations in the planner; label imagined sides as preview
GPU cost per design too high	Cache SceneGraph, render only masks, prefetch evening only for the current version

Open questions

- ☐ Can we meet the Creators API sales requirement before launch, and how long do the Associates terms let us cache prices and images?
- ☐ Accounts in MVP, or anonymous device projects until Next?
- ☐ Per-device limits on free generations, and is there a paid tier?

- ☐ Which vendor wins each model role after the bake-off?
- ☐ Does the landlord letter need legal review before launch in each market?

Milestones

Phase	Exit criteria
0. Model bake-off	200-photo eval set scored; one model chosen per role; cost and latency measured
1. Pipeline alpha	Photo → verified render + priced list for the birthday scenario, internal only
2. MVP beta	Intake, workspace, list, moods + evening, \$0 Rearrange, compare; accessibility audit passed
3. Public MVP	Latency and honesty targets met for two weeks of beta traffic
4. Next	Landlord letter, Fit to budget, grouping, accounts, saved projects